

Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura



Análisis de Lenguajes de Programación
R-322

Informe Trabajo Práctico Final
EDSL de Contratos Financieros en Haskell

Alumno
Bussanich, Santiago — Legajo: 6488/2

26 de Febrero, 2026

Índice

1. Descripción del proyecto	2
1.1. Introducción	2
1.1.1. Contratos financieros	2
1.1.2. Observables	3
1.2. Objetivo	3
2. El lenguaje	4
2.1. Sintaxis abstracta	4
2.2. Sintaxis concreta	4
2.3. Ejemplos	5
3. Manual de uso e instalación	6
3.1. Compilación	6
3.2. Ejecución del REPL	7
3.3. Uso del REPL	7
3.4. Comandos disponibles	8
3.4.1. Comandos del lenguaje	8
3.4.2. Comandos del REPL	8
3.5. Archivos <code>.fin</code>	9
4. Organización de los archivos	9
5. Decisiones de diseño y observaciones	10
5.1. El AST	10
5.2. Parsing	11
5.3. La mónada <code>Interp</code>	12
5.4. El evaluador	13
5.5. Sustitución de variables y detección de ciclos	14
5.6. Sistema de billeteras y firmas	14
5.6.1. Billeteras	14
5.6.2. Contratos pendientes y firmas	15
5.7. Evolución temporal y <code>setfecha</code>	16
5.8. Oráculo configurable	16
5.9. Pretty printer	17
5.10. Exportación y visualización	17
5.10.1. Reporte HTML	17
5.10.2. SVG del AST (via Graphviz)	18
5.10.3. Gráfico de barras	18
5.10.4. Tabla de evolución temporal	18
5.11. Evaluación con fecha mutable	18
5.12. El REPL	19
6. Conclusión	20
7. Bibliografía	20

1. Descripción del proyecto

1.1. Introducción

1.1.1. Contratos financieros

Un contrato financiero es un acuerdo legal entre dos o más partes que establece obligaciones y derechos sobre flujos de dinero futuros. Estos contratos son fundamentales en los mercados financieros y abarcan instrumentos como bonos, préstamos, swaps de monedas, opciones y derivados.

La diversidad y complejidad de los contratos financieros los convierte en un dominio ideal para ser modelado mediante lenguajes formales. De manera análoga a como los lenguajes de programación permiten describir computaciones, un lenguaje de contratos financieros permite describir de forma precisa y composicional las obligaciones monetarias entre partes.

En su paper *“Composing Contracts: An Adventure in Financial Engineering”* [1], Peyton Jones, Eber y Seward proponen modelar contratos financieros como un tipo de datos algebraico, donde contratos complejos se construyen a partir de combinadores simples. Los combinadores principales son:

- **zero**: el contrato nulo, sin obligaciones para ninguna de las partes. Es el elemento neutro de la composición: `c and zero` es equivalente a `c`. Permite representar la ausencia de un acuerdo o la rama vacía de una elección.
- **one c**: el contrato más simple con contenido: la contraparte entrega inmediatamente una unidad de la moneda `c` al titular del contrato. Es el “átomo” a partir del cual se construyen todos los demás contratos con valor monetario.
- **give c**: invierte los roles de un contrato. Si `c` dice “la contraparte me paga”, entonces `give c` dice “yo le pago a la contraparte”. Aplicar **give** dos veces devuelve el contrato original. Esto permite modelar obligaciones bidireccionales: por ejemplo, un swap se expresa como `give one USD and one EUR`.
- **and c1 c2**: adquiere ambos contratos simultáneamente. Ambas obligaciones se ejecutan y sus flujos de caja se combinan.
- **or c1 c2**: otorga al titular el derecho de elegir entre `c1` y `c2`, seleccionando el más favorable. Este combinador es fundamental para modelar opciones financieras: una opción de compra (*call*) puede expresarse como la elección entre ejercer el contrato o no hacer nada (`contrato or zero`).
- **then c1 c2**: ejecuta `c1`; si éste no produce ningún flujo de caja (por ejemplo, porque venció), utiliza `c2` como respaldo. Sirve para modelar contratos con fallback: “intentar cobrar el bono; si expiró, ejecutar la garantía”.
- **scale o c**: multiplica todos los montos del contrato `c` por el valor del observable `o`. Es lo que permite pasar de “una unidad de USD” a “1000 USD” (con `scale 1000.0 one USD`), o vincular un contrato a un precio de mercado variable (con `scale OIL one USD`).

- **truncate** *d c*: limita la vigencia del contrato *c* hasta la fecha *d*. Si la fecha actual es posterior a *d*, el contrato no genera flujos de caja (se comporta como **zero**). Es importante notar que **truncate** no programa un pago para la fecha *d*, sino que establece un horizonte de vencimiento: el contrato es válido **hasta** esa fecha.

Este enfoque composicional permite construir contratos arbitrariamente complejos a partir de piezas simples, de la misma manera en que un programa se construye a partir de expresiones y sentencias.

1.1.2. Observables

Los observables representan valores que pueden variar en el tiempo y que son consultados durante la evaluación de un contrato. Típicamente se trata de precios de mercado, tasas de cambio, o índices financieros. La idea de éstos es que representen alguna cantidad que pueda ser medida objetivamente. Por ejemplo, el valor al que cotiza el dólar en Argentina hoy puede ser buscado y verificado por ambas partes, mientras que el valor subjetivo que una persona le asigna a su propiedad puede no serlo.

En nuestro lenguaje, los observables se utilizan dentro del constructor **scale** para determinar el factor de escala que se aplica a los flujos de caja generados por un contrato, y dentro de los condicionales **if** para tomar decisiones basadas en comparaciones numéricas. Un observable puede ser una constante numérica (por ejemplo, `1000.0`), un valor externo que se obtiene de un oráculo (por ejemplo, `OIL` para el precio del petróleo), o una consulta de saldo de billetera (por ejemplo, `balance Alice USD`). Además, es posible combinar observables mediante operaciones aritméticas (+, -, *, /).

1.2. Objetivo

En este trabajo desarrollamos un **Embedded Domain Specific Language** (EDSL) implementado en Haskell para modelar, evaluar y simular contratos financieros. El lenguaje, al que llamamos *Financial Contracts EDSL*, permite:

1. **Definir** contratos financieros de forma declarativa y composicional.
2. **Parsear** descripciones de contratos a partir de texto, utilizando combinadores de Parsec.
3. **Evaluar** contratos para generar flujos de caja (*cashflows*) concretos.
4. **Componer** contratos de forma modular usando los combinadores del paper.
5. **Nombrar** contratos con **let** y reutilizarlos como variables.
6. **Pretty-printing** contratos con paréntesis mínimos respetando precedencias.
7. **Simular** la ejecución con billeteras, verificación de saldos y firmas duales.
8. **Condicionar** contratos a observables booleanos con **if/else**.
9. **Avanzar en el tiempo** con **setfecha**, observando la evolución temporal de las billeteras.
10. **Exportar** resultados a reportes HTML con tablas y gráficos CSS embebidos, y diagramas de AST en SVG.

El lenguaje se presenta con un REPL interactivo que permite definir, componer, evaluar y visualizar contratos financieros de forma inmediata. Se trata de una implementación del paper de Peyton Jones et al., con simplificaciones en la evaluación de **or** y **then**, y extensiones significativas en cuanto a billeteras, firmas, condicionales, exportación y visualización.

2. El lenguaje

2.1. Sintaxis abstracta

La sintaxis abstracta del lenguaje se divide en cuatro categorías principales: contratos, observables numéricos, observables booleanos y comandos.

```
contract ::= zero
           | one currency
           | give contract
           | contract and contract
           | contract or contract
           | contract then contract
           | scale obs contract
           | truncate date contract
           | if obsBool then contract else contract
           | variable

obs ::= number
        | identifier
        | balance party currency
        | obs + obs
        | obs - obs
        | obs * obs
        | obs / obs
        | - obs

obsBool ::= obs > obs
            | obs < obs
            | obs >= obs
            | obs <= obs
            | obs == obs

comm ::= let variable = contract
          | comm ; comm
          | contract
          | deposit party amount currency
          | propose name contract
          | sign name party
          | execute name
          | setfecha date

currency ::= USD | EUR | ARS | GBP
date ::= YYYY-MM-DD
```

2.2. Sintaxis concreta

La sintaxis concreta sigue la gramática de precedencias implementada en el parser.

```

digit ::= '0' | '1' | ... | '9'
nat   ::= digit | digit nat
float ::= nat '.' nat
identifier ::= letter rest
rest ::= letter rest | digit rest | ε

comm ::= singleComm ';' comm | singleComm
singleComm ::= assign | deposit | propose | sign | execute | setfecha | evalComm
assign ::= 'let' identifier '=' contract
deposit ::= 'deposit' identifier number currency
propose ::= 'propose' identifier contract
sign ::= 'sign' identifier identifier
execute ::= 'execute' identifier
setfecha ::= 'setfecha' date
evalComm ::= contract

contract ::= term 'or' term | term
term ::= factor 'and' factor | factor
factor ::= prefix 'then' prefix | prefix
prefix ::= 'give' prefix
| 'scale' observable prefix
| 'truncate' date prefix
| 'if' obsBool 'then' prefix 'else' prefix
| atom
atom ::= '(' contract ')' | 'zero' | 'one' currency | identifier

obsBool ::= observable boolOp observable
boolOp ::= '>' | '<' | '>=' | '<=' | '=='

observable ::= obsTerm '+' obsTerm | obsTerm '-' obsTerm | obsTerm
obsTerm ::= obsAtom '*' obsAtom | obsAtom '/' obsAtom | obsAtom
obsAtom ::= '(' observable ')' | '-' obsAtom | float | nat
| 'balance' identifier currency | identifier

date ::= digit4 '-' digit2 '-' digit2
currency ::= 'USD' | 'EUR' | 'ARS' | 'GBP'

```

2.3. Ejemplos

Para comprender mejor la semántica del lenguaje, veamos algunos ejemplos concretos. El contrato financiero más básico que podemos expresar es un *zero-coupon bond*: un instrumento que promete un pago fijo en una fecha futura.

```

1 -- Un zero-coupon bond: recibir 1000 USD antes del 2027-01-01
2 let pago = one USD;
3 let bono = scale 1000.0 pago;
4 let zcb = truncate 2027-01-01 bono;
5 zcb

```

Listing 1: Zero-coupon bond (basico.fin)

Este programa define un contrato que genera un pago de 1000 USD con vencimiento

al 1 de enero de 2027. Podemos observar varias características del lenguaje: la definición de contratos con `let`, la composición mediante `scale` y `truncate`, y la evaluación final al escribir simplemente el nombre del contrato.

Un ejemplo más interesante es un *currency swap*, donde una parte paga en una moneda y recibe en otra:

```
1 -- Un currency swap: yo pago USD y recibo EUR
2 let pagoUSD = give one USD;
3 let cobroEUR = one EUR;
4 let swap = pagoUSD and cobroEUR;
5 scale 1000.0 swap
```

Listing 2: Currency swap (swap.fin)

Aquí utilizamos `give` para invertir la dirección del pago y `and` para combinar ambas obligaciones. Finalmente, `scale 1000.0` amplifica los montos a cantidades significativas.

Un ejemplo que combina billeteras, firmas y evolución temporal:

```
1 setfecha 2026-01-01;
2 deposit Bussa 10000 USD;
3 deposit Banco 5000 USD;
4
5 setfecha 2026-06-01;
6 propose pago1 give scale 3000.0 one USD;
7 sign pago1 Bussa; sign pago1 Banco; execute pago1;
8
9 setfecha 2026-12-01;
10 propose pago2 give scale 2000.0 one USD;
11 sign pago2 Bussa; sign pago2 Banco; execute pago2
```

Listing 3: Evolución temporal con billeteras (graficoTemporal.fin)

Este ejemplo muestra el flujo completo de simulación: se fija una fecha inicial, se depositan fondos en las billeteras, se avanza en el tiempo, se proponen contratos, ambas partes firman, y se ejecutan. Al exportar un reporte HTML, se genera una tabla de eventos y una tabla de evolución por moneda que muestra cómo varían los saldos de cada parte a lo largo del tiempo.

Finalmente, un ejemplo con condicionales:

```
1 -- Derecho a comprar acciones de AAPL al precio actual
2 let compra = scale AAPL one USD;
3 -- or zero: si el contrato no conviene, no se ejerce
4 let call = compra or zero;
5 call
```

Listing 4: Opción call condicional (opcion_call.fin)

3. Manual de uso e instalación

3.1. Compilación

Para compilar el proyecto, debemos ejecutar el comando

```
stack build
```

desde el directorio raíz del proyecto. Esto descargará las dependencias necesarias (Parsec, Haskell, mtl, etc.) y compilará el intérprete.

3.2. Ejecución del REPL

Una vez compilado, podemos iniciar el REPL interactivo con:

```
stack run
```

Esto nos presenta el intérprete con el prompt `Fin>`, donde podemos escribir contratos, comandos y utilizar las directivas del sistema.

3.3. Uso del REPL

```
Intérprete de EDSL de Contratos Financieros.
Escriba :? para recibir ayuda.
Fin> one USD
Fecha          Monto          Desde -> Hacia
-----
2026-02-26    1 USD    Bob -> Alice

Fin> let swap = give one USD and one EUR
Contrato(s) asignado(s) correctamente.

Fin> scale 1000.0 swap
Fecha          Monto          Desde -> Hacia
-----
2026-02-26    1000 USD    Alice -> Bob
2026-02-26    1000 EUR    Bob -> Alice

Fin> deposit Alice 5000 USD
Contrato(s) asignado(s) correctamente.

Fin> :w Alice
[Alice]
-----
USD    5000

Fin> :l Ejemplos/basico.fin
Archivo Ejemplos/basico.fin cargado correctamente.
Fecha          Monto          Desde -> Hacia
-----
2026-02-26    1000 USD    Bob -> Alice

Fin> :r reporte.html
Reporte exportado a reporte.html

Fin> :q
Saliendo del interprete financiero.
```


3.4. Comandos disponibles

Los comandos se dividen en dos categorías: comandos del **lenguaje** (que se escriben directamente sin prefijo) y comandos del **REPL** (prefijados con `:`).

3.4.1. Comandos del lenguaje

Comando	Descripción
<code><contrato></code>	Evaluar un contrato y mostrar cashflows
<code>let x = <contrato></code>	Asignar un alias a un contrato
<code>c1 ; c2</code>	Secuenciar comandos
<code>deposit <parte><monto><moneda></code>	Depositar fondos en una billetera
<code>propose <nombre><contrato></code>	Proponer contrato (requiere firmas)
<code>sign <nombre><parte></code>	Firmar un contrato pendiente
<code>execute <nombre></code>	Ejecutar contrato firmado
<code>setfecha YYYY-MM-DD</code>	Cambiar fecha de evaluación
<code>if <cond>then <c>else <c></code>	Condicional sobre observables

3.4.2. Comandos del REPL

Cada comando del REPL tiene un alias corto que permite invocarlo más rápidamente. La resolución de comandos prioriza coincidencias exactas contra los alias antes de buscar por prefijo, lo que evita ambigüedades.

Comando	Alias	Descripción
<code>:ast <exp></code>	<code>:a</code>	Mostrar el AST de un contrato
<code>:pp <exp></code>		Pretty print de un contrato
<code>:eval <exp></code>	<code>:e</code>	Evaluar un contrato directamente
<code>:load <archivo></code>	<code>:l</code>	Cargar un archivo <code>.fin</code>
<code>:store</code>	<code>:s</code>	Mostrar contratos definidos
<code>:quote [<nombre><val>]</code>	<code>:qt</code>	Ver/definir cotización del oráculo
<code>:partes [<yo><contra>]</code>	<code>:p</code>	Ver/cambiar las partes del contrato
<code>:wallet [<parte>]</code>	<code>:w</code>	Ver billetera(s)
<code>:pending</code>	<code>:pd</code>	Ver contratos pendientes de firma
<code>:setfecha <YYYY-MM-DD></code>	<code>:sf</code>	Cambiar fecha de evaluación
<code>:historial</code>	<code>:hi</code>	Ver historial de ejecuciones
<code>:reporte [<archivo.html>]</code>	<code>:r</code>	Exportar reporte HTML completo
<code>:ast-svg <contrato> '>' <archivo></code>	<code>:svg</code>	Exportar AST del contrato a SVG
<code>:quit</code>	<code>:q</code>	Salir del intérprete
<code>:help o :?</code>	<code>:h</code>	Mostrar ayuda

3.5. Archivos `.fin`

Es posible escribir programas en archivos con extensión `.fin` y cargarlos desde el REPL con `:load`. El directorio `Ejemplos/` contiene 15 archivos de ejemplo que cubren distintos escenarios financieros. La sintaxis de estos archivos sigue las siguientes convenciones:

- Los comentarios se escriben con `--` al inicio de la línea.
- Las líneas vacías se ignoran.
- Los comandos se separan explícitamente con `;`.
- La última expresión (la evaluación) no lleva `;` al final.

4. Organización de los archivos

El trabajo se encuentra dividido en los siguientes directorios principales:

`src/`. Contiene el código fuente principal del lenguaje:

- **AST.hs**. Define los tipos de datos algebraicos del lenguaje: `Contract`, `Obs`, `ObsBool` y `Comm`.
- **PendingContracts.hs**. Define el tipo `PendingContract` y las operaciones asociadas: creación, firma y verificación de contratos pendientes.
- **Types.hs**. Define los tipos base: `Currency`, `Amount`, `Cashflow`, `Env`, `Wallets`, `HistorialEntry`, `WalletSnapshot` y `EvalError`.
- **Monads.hs**. Define la mónada `Interp` (un stack de `StateT`, `ReaderT`, `WriterT` y `Either` de `mtl`) junto con el estado mutable `InterpState` y las funciones de acceso.
- **Parser.hs**. Implementa el parser del lenguaje utilizando `Parsec`, con gramática de precedencias para contratos, observables numéricos, observables booleanos y todos los comandos.
- **Evaluator.hs**. Implementa el evaluador de contratos, observables, observables booleanos y comandos (incluyendo `deposit`, `propose`, `sign`, `execute`, `setfecha`), además de las operaciones de billetera.
- **PrettyPrinter.hs**. Pretty printer con niveles de precedencia, formateo de cashflows, billeteras, historial, pendientes y errores.
- **Exportar.hs**. Genera reportes HTML autocontenidos con barras de cashflows en HTML+CSS y tablas de evolución temporal de billeteras.
- **ExportarAST.hs**. Genera diagramas de árbol SVG del AST de contratos, convirtiendo el árbol a formato DOT y delegando el layout y renderizado a `Graphviz`[8].

`app/`. Contiene un único archivo, `Main.hs`, donde se encuentra el REPL interactivo construido con `Haskeline`. En este archivo se definen los comandos del intérprete, el manejo de estado (`InterpState + Env`), la carga de archivos `.fin`, y la interfaz de usuario.

`Ejemplos/`. Es el directorio donde se almacenan los 15 archivos de ejemplo `.fin` que cubren escenarios como bonos, swaps, opciones, derivados, coberturas, préstamos y evolución temporal.

`Exports/`. Contiene ejemplos pre-generados de las salidas del sistema: reportes HTML completos (`reporte_completo.html`, `reporte_empresa.html`, `reporte_temporal.html`) y diagramas SVG de árboles AST (`ast_bono.svg`, `ast_collar.svg`, `ast_opcion.svg`, etc.). Estos archivos permiten visualizar las capacidades del sistema sin necesidad de compilarlo.

Los demás archivos y directorios que pueden encontrarse en el directorio raíz (`stack.yaml`, `package.yaml`, `.stack-work/`, etc.) son generados y utilizados por la herramienta `Stack`.

5. Decisiones de diseño y observaciones

5.1. El AST

Los tipos de datos del lenguaje están definidos en `AST.hs` como tipos algebraicos de Haskell siguiendo un enfoque de **deep embedding**: los contratos se representan como un árbol de sintaxis abstracta (AST) que puede ser inspeccionado, transformado y evaluado. A diferencia de un *shallow embedding*, donde cada combinador se definiría directamente como una función Haskell que computa su resultado, el deep embedding nos permite operar sobre la *estructura* del contrato. Esto es lo que hace posible, por ejemplo, hacer pretty printing, sustituir variables, detectar ciclos, exportar diagramas SVG del AST, y potencialmente aplicar optimizaciones o transformaciones sobre contratos antes de evaluarlos.

El tipo principal `Contract` representa los combinadores del paper, extendidos con condicionales:

```
1 data Contract
2   = Zero
3   | One Currency
4   | Give Contract
5   | And Contract Contract
6   | Or  Contract Contract
7   | Truncate Date Contract
8   | Then Contract Contract
9   | Scale (Obs Double) Contract
10  | Var String
11  | If ObsBool Contract Contract
12  deriving (Show, Eq)
```

Listing 6: Tipo de datos Contract (AST.hs)

El constructor `Var String` no forma parte del paper original, sino que fue añadido para soportar el sistema de variables (aliasing). Cuando el usuario escribe `let swap = give one USD and one EUR`, el parser genera un `Assign "swap"(And (Give (One USD)) (One EUR))`, y posteriores referencias a `swap` se parsean como `Var "swap"`.

El constructor `If ObsBool Contract Contract` permite condicionar la elección de contrato en base a comparaciones numéricas de observables. Por ejemplo, `if OIL >100.0 then scale OIL one USD else zero` evalúa un contrato u otro según el precio del petróleo.

Los observables booleanos se definen como un tipo separado para mantener la separación de tipos limpia:

```
1 data ObsBool
2   = Gt  (Obs Double) (Obs Double)  -- obs > obs
3   | Lt  (Obs Double) (Obs Double)  -- obs < obs
4   | Gte (Obs Double) (Obs Double)  -- obs >= obs
5   | Lte (Obs Double) (Obs Double)  -- obs <= obs
6   | Eq  (Obs Double) (Obs Double)  -- obs == obs
7   deriving (Show, Eq)
```

Listing 7: Tipo de datos ObsBool (AST.hs)

Los comandos se representan con el tipo `Comm`, que incluye todos los comandos del lenguaje:

```

1 data Comm
2   = Assign  String Contract
3   | Seq     Comm Comm
4   | Run     Contract
5   | Deposit PartyId Currency Double
6   | Propose String Contract
7   | Sign    String PartyId
8   | Execute String
9   | SetFecha Date
10  deriving (Show, Eq)

```

Listing 8: Tipo de datos Comm (AST.hs)

Finalmente, los observables numéricos se representan como un tipo paramétrico, extendidos con un constructor **Balance** para consultar saldos de billetera:

```

1 data Obs a
2   = Konst      a
3   | Add        (Obs a) (Obs a)
4   | Sub        (Obs a) (Obs a)
5   | Mul        (Obs a) (Obs a)
6   | Div        (Obs a) (Obs a)
7   | Neg        (Obs a)
8   | External   String
9   | Balance    PartyId Currency
10  deriving (Show, Eq, Read)

```

Listing 9: Tipo de datos Obs (AST.hs)

El constructor **Balance PartyId Currency** permite que un contrato consulte dinámicamente el saldo de una parte en una moneda determinada. Esto habilita contratos como “si Alice tiene más de 1000 USD, entonces pagar 500 EUR”.

Hemos implementado las instancias de **Num** y **Fractional** para **Obs a**, lo cual permite escribir expresiones aritméticas directamente sobre observables utilizando la sintaxis natural de Haskell a nivel de la implementación.

5.2. Parsing

Para el parseo utilizamos la librería **Parsec** [2], que ya habíamos utilizado en otros trabajos de la materia. La sintaxis del lenguaje es relativamente simple y alternativas como **Happy** resultaban innecesariamente complejas para nuestro alcance.

La gramática de contratos se implementó con una jerarquía de precedencias para resolver la ambigüedad de los operadores infijos:

Precedencia	Operador	Asociatividad
0 (más baja)	or	izquierda
1	and	izquierda
2	then	izquierda
3	prefijos (give, scale, truncate, if)	—
4 (más alta)	átomos (zero, one, variables, paréntesis)	—

Esta jerarquía se implementa con funciones mutuamente recursivas utilizando `chainl1` de Parsec:

```
1 parserContract :: Parser Contract
2 parserContract = chainl1 parserTerm
3                 (reserved "or" >> return Or)
4
5 parserTerm :: Parser Contract
6 parserTerm = chainl1 parserFactor
7             (reserved "and" >> return And)
8
9 parserFactor :: Parser Contract
10 parserFactor = chainl1 parserPrefix
11              (reserved "then" >> return Then)
```

Listing 10: Jerarquía de precedencias en el parser

De manera similar, los observables tienen su propia jerarquía: suma/resta en el nivel más bajo, multiplicación/división en el nivel intermedio, y átomos (constantes, externos, balance, paréntesis) en el nivel más alto.

El parser de comandos usa `try` para intentar cada tipo de comando en orden, dándole prioridad a los que comienzan con palabras reservadas (`let`, `deposit`, `propose`, etc.) antes de intentar parsear un contrato como evaluación directa.

5.3. La mónada Interp

La mónada de evaluación se construye como un stack de transformadores de mónadas de la librería `mtl`:

```
1 type Interp = StateT InterpState
2               (ReaderT Env
3                (WriterT [Cashflow]
4                 (Either EvalError)))
```

Listing 11: Definición de la mónada Interp (Monads.hs)

Esta mónada combina cuatro efectos:

- **State** (`InterpState`): estado mutable que contiene el store de contratos, las billeteras, los contratos pendientes, las cotizaciones del oráculo, el historial de ejecuciones, los snapshots temporales y la fecha actual.
- **Reader** (`Env`): acceso de solo lectura al entorno de evaluación, que contiene la función oráculo y las identidades de las partes del contrato.
- **Writer** (`[Cashflow]`): acumulación de flujos de caja generados durante la evaluación.
- **Either** (`EvalError`): manejo de errores (división por cero, observables desconocidos, saldo insuficiente, etc.).

El estado mutable `InterpState` agrupa toda la información que evoluciona durante la sesión:

```

1 data InterpState = InterpState
2   { isContracts :: ContractStore
3   , isWallets    :: Wallets
4   , isPending    :: PendingStore
5   , isQuotes     :: Map String Double
6   , isHistorial  :: [HistorialEntry]
7   , isSnapshots  :: [WalletSnapshot]
8   , isFechaHoy   :: Date
9   } deriving (Show)

```

Listing 12: Estado mutable del intérprete (Monads.hs)

Las funciones de acceso (`getContracts`, `putWallets`, `appendHistorial`, `appendSnapshot`, etc.) se definen como operaciones monádicas simples sobre este estado.

5.4. El evaluador

El evaluador se divide en cuatro funciones principales:

`evalContract`. Evalúa un contrato dentro de la mónada `Interp`, generando cashflows. Los casos más interesantes son:

- **Give**: utiliza `localEnv` para intercambiar `yo` y `contraparte` en el entorno, invirtiendo la dirección de los pagos.
- **Or**: evalúa ambas ramas y elige la que maximiza el valor neto para la parte “yo”, convirtiendo montos a USD mediante el oráculo para comparar. Si una rama falla y la otra no, se elige la exitosa.
- **Then**: evalúa la primera rama; si no genera cashflows, pasa a la segunda (mecanismo de fallback).
- **Scale**: evalúa el observable, y usa `sensorFlows` para multiplicar los montos de todos los cashflows por el factor obtenido.
- **Truncate**: usa `sensorFlows` para filtrar los cashflows cuya fecha exceda el límite.
- **If**: evalúa el observable booleano y, según el resultado, evalúa la rama verdadera o la falsa.

`evalObs`. Evalúa un observable numérico. Soporta constantes, valores externos (obtenidos mediante la función oráculo del entorno), consultas de `Balance` al estado de billeteras, y las cuatro operaciones aritméticas. La división por cero y el uso de observables no definidos en el oráculo se detectan y reportan como error.

`evalBool`. Evalúa un observable booleano comparando dos observables numéricos con los operadores `>`, `<`, `>=`, `<=` o `==`.

`evalComm`. Evalúa un comando, actualizando el `InterpState`. Los comandos se manejan así:

- **Assign**: almacena el contrato en el store **sin resolver variables**; la resolución se difiere al momento de la evaluación.
- **Run**: resuelve las variables con `substContract` y evalúa el contrato.
- **Seq**: ejecuta secuencialmente, propagando el estado actualizado.

- **Deposit**: agrega fondos a la billetera de una parte y registra un snapshot temporal.
- **Propose**: crea un contrato pendiente de firma entre las partes actuales.
- **Sign**: registra la firma de una parte sobre un contrato pendiente.
- **Execute**: si el contrato está completamente firmado, resuelve y evalúa el contrato, aplica los cashflows a las billeteras (verificando saldos), y registra la ejecución en el historial.
- **SetFecha**: cambia la fecha de evaluación y registra un snapshot del estado actual de las billeteras.

5.5. Sustitución de variables y detección de ciclos

El sistema de variables se implementa mediante la función `substContract`, que recorre recursivamente el contrato y reemplaza cada `Var name` por el contrato correspondiente del `ContractStore`. Esta resolución se realiza **antes** de la evaluación (al ejecutar un `Run` o `Execute`), lo que permite mantener la mónada `Interp` simple.

Para evitar ciclos infinitos (por ejemplo, `let x = x` o cadenas indirectas como `let a = b; let b = a`), la función mantiene un `Set String` con las variables ya visitadas:

```

1 substContract :: ContractStore -> Contract
2               -> Either EvalError Contract
3 substContract store = go Set.empty
4   where
5     go visited (Var name)
6       | Set.member name visited =
7         Left (EvalMsg $
8             "Referencia ciclica: " ++ name)
9       | otherwise =
10        case Map.lookup name store of
11          Just c  ->
12            go (Set.insert name visited) c
13          Nothing ->
14            Left (EvalMsg $
15                "Contrato no definido: " ++ name)
16    go visited (Give c)      =
17      Give <$> go visited c
18    go visited (And c1 c2) =
19      And <$> go visited c1 <*> go visited c2
20    -- ... (analogamente para los demas)

```

Listing 13: Detección de ciclos en `substContract` (Evaluator.hs)

Es importante notar que el comando `Assign` almacena el contrato tal cual, sin resolver variables. Esto permite que las definiciones sean “lazy”: un contrato puede hacer referencia a variables que aún no fueron definidas al momento de la asignación, siempre que estén definidas cuando se evalúe.

5.6. Sistema de billeteras y firmas

5.6.1. Billeteras

Cada parte del contrato tiene una billetera representada como un `Map Currency Double`, que registra su saldo en cada moneda. El tipo global es:

```
1 type Wallets = Map PartyId (Map Currency Double)
```

Listing 14: Tipo de billeteras (Types.hs)

Las operaciones básicas son `walletDeposit` (agregar fondos), `walletWithdraw` (debitar con verificación de saldo) y `applyWalletCashflows` (aplicar una lista de flujos de caja a las billeteras, verificando que cada pagador tenga fondos suficientes). Si un pagador no tiene saldo suficiente, la operación falla con un mensaje de error indicando el faltante.

5.6.2. Contratos pendientes y firmas

Para ejecutar un contrato a través del sistema de billeteras, se requiere que ambas partes lo firmen explícitamente. El flujo es:

1. `propose <nombre><contrato>`: propone un contrato entre las partes actuales. Se crea un `PendingContract` con ambas firmas en estado `Pending`.
2. `sign <nombre><parte>`: la parte indicada firma el contrato. Solo las partes involucradas pueden firmar.
3. `execute <nombre>`: si ambas partes firmaron, el contrato se evalúa y los flujos de caja resultantes se aplican a las billeteras. Si el pagador no tiene fondos suficientes, la ejecución falla sin modificar las billeteras.

El tipo `PendingContract` almacena el contrato, las partes, el estado de firma de cada una, y la fecha de creación:

```
1 data PendingContract = PendingContract
2   { pcName      :: String
3   , pcContract  :: Contract
4   , pcPartyA    :: PartyId
5   , pcPartyB    :: PartyId
6   , pcSignA     :: SignatureStatus
7   , pcSignB     :: SignatureStatus
8   , pcCreatedAt :: Day
9   } deriving (Show)
```

Listing 15: Contrato pendiente (PendingContracts.hs)

```
Fin> :partes Alice Bob
Partes actualizadas: yo = Alice, contraparte = Bob

Fin> deposit Alice 5000 USD; deposit Bob 3000 EUR

Fin> propose miSwap
      scale 1000.0 (give one USD and one EUR)

Fin> sign miSwap Alice; sign miSwap Bob

Fin> execute miSwap
Fecha      Monto      Desde -> Hacia
-----
2026-02-26 1000 USD   Alice -> Bob
2026-02-26 1000 EUR   Bob -> Alice
Billeteras actualizadas:
[Alice]
```


USD	4000
EUR	1000
[Bob]	
USD	1000
EUR	2000

Listing 16: Sesión con billeteras y firmas

5.7. Evolución temporal y setfecha

El comando `setfecha` permite cambiar la fecha de evaluación durante la sesión. Esto posibilita simular escenarios temporales: depositar fondos en una fecha, ejecutar contratos en otra, y observar cómo evolucionan las billeteras.

Cada vez que se ejecuta `setfecha`, `deposit` o `execute`, se registra un `WalletSnapshot` con la fecha, el estado de las billeteras y una descripción del evento:

```

1 data WalletSnapshot = WalletSnapshot
2   { snapFecha      :: Day
3   , snapWallets    :: Wallets
4   , snapEvento     :: String
5   } deriving (Show, Eq, Generic)

```

Listing 17: Snapshot temporal (Types.hs)

Estos snapshots se utilizan al exportar el reporte HTML para generar la sección de evolución temporal: una tabla de eventos y una tabla de saldos por moneda que muestra la evolución del saldo de cada parte a lo largo del tiempo.

5.8. Oráculo configurable

El entorno de evaluación incluye una función `getQuote :: String -> Maybe Double` que actúa como oráculo para los observables externos. En lugar de utilizar una función hardcodeada, el oráculo se construye a partir de un mapa de cotizaciones (`Map String Double`) almacenado en el estado del REPL:

```

1 makeOracle :: Map String Double
2   -> (String -> Maybe Double)
3 makeOracle quotes name = Map.lookup name quotes

```

Listing 18: Construcción del oráculo (Main.hs)

El comando `:quote` permite consultar y modificar las cotizaciones en tiempo de ejecución. Al actualizar una cotización, se reconstruye la función oráculo y se actualiza el entorno. El REPL inicializa el oráculo con valores por defecto:

Observable	Valor
OIL	80.5
AAPL	150.0
USD_ARS	1000.0
EUR_USD	1.08
GBP_USD	1.27
ARS_USD	0.001

5.9. Pretty printer

El pretty printer utiliza niveles de precedencia para minimizar el uso de paréntesis. Cada operador tiene asociado un nivel de precedencia, y los paréntesis solo se insertan cuando el nivel de precedencia del contexto exterior es mayor al del operador actual.

Prec.	Contratos
0	or
1	and
2	then
3	prefijos (give, scale, truncate, if)
4	átomos (zero, one, variables)

Prec.	Observables
0	+, -
1	*, /
2	átomos (constantes, externos, balance)

De esta manera, una expresión como (one USD and give one EUR) or zero se imprime como one USD and give one EUR or zero, omitiendo los paréntesis innecesarios gracias a que and tiene mayor precedencia que or.

5.10. Exportación y visualización

El sistema ofrece dos principales opciones en cuanto a la exportación y visualización, las cuales son un **reporte HTML** completo y un archivo SVG (de sus siglas **Scalable Vector Graphics**) para una mejor **visualización del AST**. El archivo SVG se genera mediante la librería **Graphviz**. Una decisión personal fue evitar lo máximo posible el uso del formato SVG directamente en el código HTML por una simple razón: este formato introduce muchas líneas de código referentes al cálculo de coordenadas, lo cual hace el mismo difícilmente legible. Para las estadísticas mostradas en el reporte HTML, pudimos modelar los gráficos necesarios con HTML+CSS puro (a costa de perder un gráfico de evolución temporal de billeteras). En lo que respecta a la visualización del AST, la librería Graphviz hace el trabajo del cálculo de coordenadas por nosotros.

5.10.1. Reporte HTML

El comando `:reporte` genera un archivo HTML autocontenido (sin dependencias externas) que incluye:

1. **Contratos definidos**: tabla con nombre y definición de cada contrato en el store.
2. **Billeteras**: tarjetas con el saldo de cada parte por moneda, coloreado en verde (positivo) o rojo (negativo).
3. **Historial de ejecuciones**: para cada ejecución, una tarjeta con los cashflows en tabla y un gráfico de barras bidireccionales en HTML+CSS embebido inline.
4. **Evolución temporal**: una tabla de eventos (depósitos, setfecha, ejecuciones) y una tabla de saldos por moneda, mostrando cómo evoluciona el saldo de cada parte a lo largo del tiempo con valores coloreados (verde positivo, rojo negativo).

Todo el CSS está inline en el HTML, lo que lo hace completamente portable.

5.10.2. SVG del AST (via Graphviz)

El comando `:ast-svg` genera un diagrama SVG del árbol de sintaxis abstracta de un contrato, utilizando Graphviz[8] para el layout y renderizado. La generación se realiza en dos fases:

1. **Generación de DOT.** La función `buildDot` recorre recursivamente el `Contract` y genera un string en formato DOT (el lenguaje de descripción de grafos de Graphviz). Cada nodo del AST recibe un identificador único (`n0`, `n1`, ...) asignado mediante un contador que se propaga a través de la recursión. Para nodos con información adicional (como el observable en `scale` o la fecha en `truncate`), se utilizan HTML labels con dos líneas de texto de diferente tamaño.
2. **Renderizado externo.** El string DOT se envía al programa `dot` (de Graphviz) vía `readProcess`, que calcula automáticamente las posiciones de los nodos y genera el SVG final. Si Graphviz no está instalado, se guarda el archivo DOT como fallback.

Cada tipo de combinador tiene un color distintivo: azul (`and`), rojo (`give`), verde (`one`), naranja (`then`), púrpura (`or`), cyan (`scale`), verde oscuro (`truncate`), magenta (`if`) y gris (`zero`).

Al delegar el layout a Graphviz, el código de `ExportarAST.hs` se reduce a la conversión del AST a DOT y a la invocación del proceso externo, eliminando la necesidad de implementar manualmente el cálculo de posiciones y el renderizado SVG.

5.10.3. Gráfico de barras

Cada entrada del historial en el reporte HTML incluye un gráfico de barras bidireccionales implementado enteramente con HTML y CSS, compuesto por dos partes:

1. **Barras por cashflow.** Para cada flujo de caja se genera una fila con una etiqueta (monto, moneda, dirección) y una barra horizontal. El contenedor se divide en dos mitades: la izquierda muestra egresos (barras rojas creciendo hacia la izquierda) y la derecha muestra ingresos (barras verdes creciendo hacia la derecha). El ancho de cada barra se normaliza como porcentaje del valor absoluto máximo.
2. **Resumen neto por moneda.** Se acumulan los flujos por moneda y se muestra una barra horizontal por cada moneda con su saldo neto, coloreada en verde o rojo según el signo.

5.10.4. Tabla de evolución temporal

Cuando existen snapshots temporales, el reporte HTML incluye una sección de evolución compuesta por:

1. **Tabla de eventos.** Lista cronológica de todos los depósitos, cambios de fecha y ejecuciones registrados durante la sesión, con columnas de fecha y descripción del evento.
2. **Tabla de saldos por moneda.** Para cada moneda utilizada en la sesión se genera una tabla HTML con una columna por cada parte y una fila por cada snapshot temporal. Los valores positivos se muestran en verde y los negativos en rojo, utilizando las clases CSS `pos` y `neg`.

5.11. Evaluación con fecha mutable

Una decisión de diseño significativa es el uso de una **fecha mutable** mediante el comando `setfecha`. La fecha se almacena en `InterpState` (campo `isFechaHoy`) y puede

cambiar durante la sesión, lo que permite simular escenarios temporales. Esta fecha se lee en `evalContract` mediante `getFechaHoy` y afecta directamente a los combinadores `truncate` (que compara contra la fecha actual) y `one` (que asigna la fecha actual a los cashflows generados).

Esto permite escribir scripts como:

```
setfecha 2026-01-01;
deposit Alice 10000 USD;
setfecha 2026-06-01;
execute pago1;
setfecha 2026-12-01;
execute pago2
```

Listing 19: Simulación temporal

donde cada ejecución ocurre en una fecha diferente, y los gráficos de evolución muestran correctamente la progresión temporal.

Sin embargo, el sistema sigue teniendo limitaciones respecto al paper original:

- **Ausencia de when/until.** No hay mecanismo para “esperar” a que una condición se cumpla en el futuro. El condicional `if` evalúa la condición en el instante actual.
- **Semántica simplificada de or.** Elige por valor neto inmediato, sin considerar el valor temporal del dinero ni pagos futuros descontados.
- **Observables constantes por instante.** Cada observable tiene un valor fijo en cada momento (configurable vía `:quote`), sin simulación de caminos estocásticos.
- **truncate como horizonte.** Solo verifica si la fecha actual es anterior al límite; no programa pagos para fechas específicas.

Para extender el sistema con soporte temporal completo, sería necesario incorporar un modelo de evolución discreta del tiempo (retículos binomiales[9] o simulaciones de Monte Carlo[10]), observables dependientes del tiempo, y los combinadores `when` y `until` con su semántica de espera.

5.12. El REPL

El REPL se implementa con la librería `Haskeline`[3], que proporciona edición de línea, historial, y autocompletado. El estado del REPL es una tupla (`InterpState`, `Env`):

- **InterpState:** todo el estado mutable del lenguaje (contratos, billeteras, pendientes, cotizaciones, historial, snapshots, fecha).
- **Env:** el entorno de solo lectura (función oráculo, identidades de las partes).

Las identidades de las partes (`yo` y `contraparte`) son configurables en tiempo de ejecución mediante `:partes`. La fecha se actualiza tanto desde el lenguaje (`setfecha`) como desde el REPL (`:setfecha`), manteniéndose sincronizada entre `InterpState` y `Env`.

El sistema de comandos distingue entre directivas (prefijadas con `:`) y expresiones directas. Las expresiones se parsean como `Comm` y se evalúan con `evalComm` a través de la mónada `Interp`, mientras que las directivas realizan acciones especiales como exportar reportes, mostrar el AST, o modificar el oráculo.

Cada comando del REPL define uno o más alias (por ejemplo, `:q` para `:quit`, `:e` para `:eval`, `:l` para `:load`). La resolución de comandos utiliza un esquema de dos fases:

primero se busca una coincidencia exacta contra los alias definidos, y solo si no se encuentra se recurre a coincidencia por prefijo (`isPrefixOf`). Este diseño evita ambigüedades; por ejemplo, `:q` resuelve exactamente a `:quit` sin confundirse con `:quote`, a pesar de que `:q` es prefijo de ambos.

6. Conclusión

En este trabajo implementamos un EDSL en Haskell para modelar, evaluar y simular contratos financieros, tomando como base los combinadores propuestos por Peyton Jones, Eber y Seward. El resultado es un lenguaje composicional con un REPL interactivo que permite definir contratos de forma declarativa, evaluarlos para obtener flujos de caja concretos, y simular escenarios completos con billeteras, firmas duales y evolución temporal.

A lo largo del desarrollo pudimos observar cómo las abstracciones de Haskell se adecuan naturalmente a este dominio: los tipos algebraicos permiten representar la estructura del AST de forma clara y extensible; los transformadores de mónadas proveen una forma modular de combinar estado mutable, entorno de solo lectura, acumulación de resultados y manejo de errores.

El sistema presenta limitaciones respecto al paper original —principalmente la ausencia de los combinadores `when` y `until`, y la evaluación simplificada de `or` sin considerar el valor temporal del dinero— pero las extensiones incorporadas (billeteras con verificación de saldo, sistema de propuestas y firmas, simulación temporal con `setfecha`, exportación a reportes HTML y diagramas SVG del AST) ofrecen una herramienta funcional para explorar y visualizar contratos financieros de forma interactiva.

7. Bibliografía

Referencias

- [1] S. Peyton Jones, J-M. Eber, J. Seward. “*Composing Contracts: An Adventure in Financial Engineering*”. ICFP 2000.
<https://www.microsoft.com/en-us/research/publication/composing-contracts-an-adventure-in-financial-engineering/>
- [2] Hackage. Documentación de Parsec.
<https://hackage.haskell.org/package/parsec>
- [3] Hackage. Documentación de Haskeline.
<https://hackage.haskell.org/package/haskeline>
- [4] Hackage. Documentación de Data.Map (containers).
<https://hackage.haskell.org/package/containers>
- [5] Hackage. Documentación de Data.Time.
<https://hackage.haskell.org/package/time>
- [6] Hackage. Documentación de mtl (Monad Transformer Library).
<https://hackage.haskell.org/package/mtl>

- [7] W3C. *Scalable Vector Graphics (SVG) 2*. W3C Candidate Recommendation, 2018.
<https://www.w3.org/TR/SVG2/>
- [8] AT&T Labs Research. *Graphviz — Graph Visualization Software*.
<https://graphviz.org/>
- [9] J. Cox, S. Ross, M. Rubinstein. “*Option Pricing: A Simplified Approach*”. Journal of Financial Economics, 1979.
https://en.wikipedia.org/wiki/Binomial_options_pricing_model
- [10] P. Boyle. “*Options: A Monte Carlo Approach*”. Journal of Financial Economics, 1977.
https://en.wikipedia.org/wiki/Monte_Carlo_methods_for_option_pricing